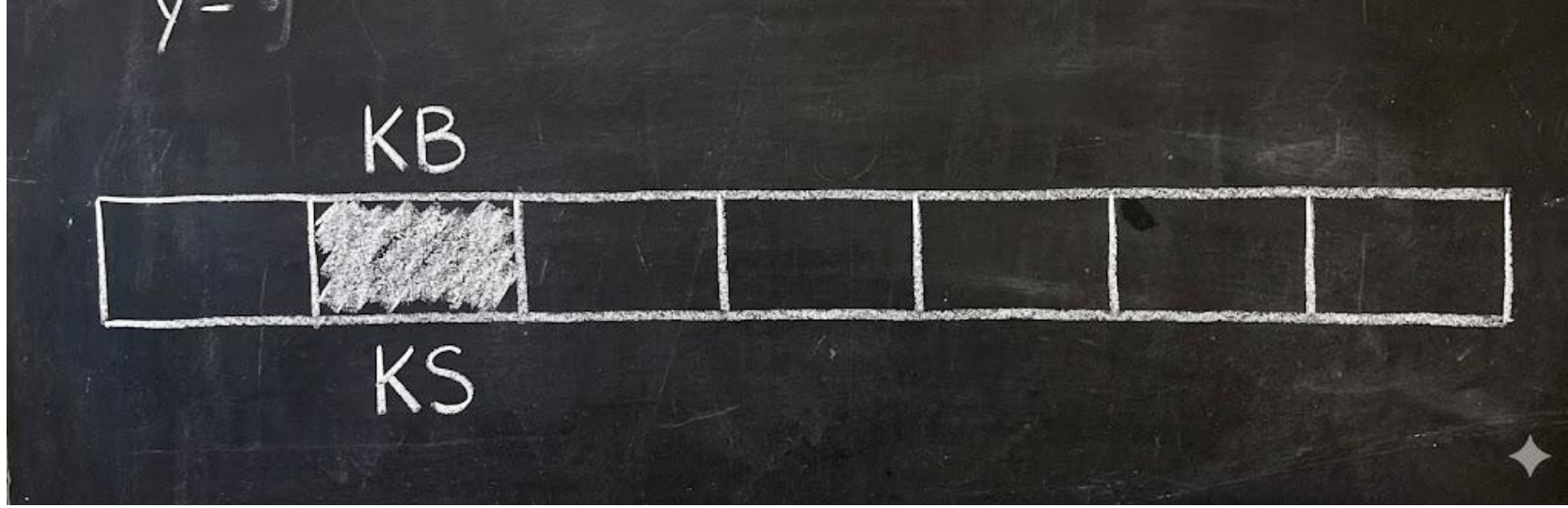


Kuyruk Veri Yapısı

Dr. Öğr. Üyesi Fatma AKALIN

Erişimin sadece iki uçtan yapılabileceği ve gelen istekleri bilginin geliş sırasına göre en önce gelen elemana ilk hizmetin verildiği veri yapılarına **ilk gelen ilk çıkar (first in first out) kuyruk veri yapıları** denir.

Kuyruk veri yapısında temel iki işlem bulunur. Bunlar kuyruğa birini almak ve kuyruğun başında bulunanı çıkarmaktır. Bu tanıma uygun bileşenler aşağıda verilmiştir.



Kuyruk yapısının olmazsa olmaz iki adet fonksiyonu bulunur.

enqueue(veri)->verilen veriyi sıraya koyar

dequeue(veri)-> sıradan bir veri çıkararak sıranın boyutunu azaltır ve çıkarılan veriyi çağrıldığı yere döndürür.

Komutlar	Sıranın durumu
enqueue(10))	10
enqueue(20))	10 -> 20
enqueue(30))	10 -> 20 ->30
dequeue() dequeue()	20 -> 30 30

Yanda gösterilen hiyerarşide bir bağlı liste üzerinden verilerin nasıl eklenip çıkarıldığı tasvir edilmiştir. Bir sıra için bağlı liste kullanılabileceği gibi dizi yapısında kullanılabilir.

Linear Queue (Lineer Kuyruk)

```
#include <iostream>
using namespace std;
#define SIZE 5

struct LinearQueue {
int data[SIZE];
int front, rear;
int cnt;
};

void initialize(LinearQueue *q) {
q->front = 0;
q->rear = -1;
q->cnt = 0;
}

bool isFull(LinearQueue *q) {
return q->cnt == SIZE;
}

bool isEmpty(LinearQueue *q) {
return q->cnt == 0;
}
```

```
void enqueue(LinearQueue *q, int val) {
if(isFull(q)) {
cout << "Kuyruk dolu" << endl;
return;
}
q->rear++;
q->data[q->rear] = val;
q->cnt++;
cout << val << " kuyruğa eklendi" << endl;
}

void dequeue(LinearQueue *q) {
if(isEmpty(q)) {
cout << "Kuyruk boş" << endl;
return;
}
int x = q->data[q->front];
q->front++;
q->cnt--;
cout << x << " kuyruktan silindi" << endl;
}
```

```
void print(LinearQueue *q) {
if(isEmpty(q)) {
cout << "Kuyruk boş" << endl;
return;
}
for(int i = q->front; i <= q->rear; i++) {
cout << q->data[i] << " ";
}
cout << endl;
}

int main() {
LinearQueue q;
initialize(&q);
enqueue(&q, 10);
enqueue(&q, 20);
enqueue(&q, 30);
print(&q);
dequeue(&q);
enqueue(&q, 40);
print(&q);
}
```

LinearQueue q;
Bu bir struct
değişkeni
pointer bekliyor
Bu yüzden
&q // adres
gönderiyoruz

```

void enqueue(LinearQueue *q, int val) {
    // 1. GERÇEK TAŞMA KONTROLÜ
    // Eğer kuyruk tamamen doluysa (boş yer yoksa)
    if (q->cnt == q->size) {
        cout << "Hata: Kuyruk tamamen dolu (Gerçek Taşma)!" << endl;
        return;
    }

    // 2. YALANCI DOLULUK (RESETLEME) KONTROLÜ
    // Rear sona dayandı ama kuyrukta boş yer var (cnt < size)
    if (q->rear == q->size - 1 && q->cnt < q->size) {
        cout << "Uyarı: Yalancı doluluk saptandı, kuyruk resetleniyor..." << endl;

        // Eğer kuyrukta hiç eleman kalmadıysa (cnt 0 ise) doğrudan başa çek
        if (q->cnt == 0) {
            q->front = 0;
            q->rear = -1;
        }
        else {
            // Eğer eleman varsa, mevcut elemanları dizinin en başına kaydır (Shifting)
            int elemanSayisi = q->cnt;
            for (int i = 0; i < elemanSayisi; i++) {
                q->data[i] = q->data[q->front + i];
            }
            q->front = 0;
            q->rear = elemanSayisi - 1;
        }
    }

    // 3. ELEMAN EKLEME
    q->rear++;
    q->data[q->rear] = val;
    q->cnt++;
    cout << val << " kuyruğa başarıyla eklendi." << endl;
}

```

**enqueue için
güvenli kontrol
yolları**

dequeue için güvenli kontrol yolları

front sona gelince resetler
ama **veri kaybı / yanlış davranış riski vardır**
Çünkü bu yapı **circular değildir**

```
void dequeue(LinearQueue *q) {  
    if (isEmpty(q)) {  
        cout << "Kuyruk boş" << endl;  
        return;  
    }  
  
    int x = q->data[q->front];  
  
    q->front++;  
    q->cnt--;  
  
    cout << x << " kuyruktan silindi" << endl;  
  
    if (q->cnt == 0) {  
        q->front = 0;  
        q->rear = -1;  
    }  
  
    // TAŞMAYI ÖNLEME KONTROLÜ  
    if (q->front >= q->size) {  
        q->front = 0;  
        q->rear = -1;  
        q->cnt = 0;  
    }  
}
```

RESETLEME!!

Yöntem	Taşma riski	Doğruluk	Öneri
Linear + reset	Var	Orta	Eğitim için
Linear + sınır kontrol	Var	Zayıf	Geçici çözüm
Circular queue	Yok	Doğru	En iyi çözüm

Linear + sınır kontrol **fiziksel taşmayı engeller**

Ama **mantıksal taşmayı (false full)** engelleyemez

Bu yüzden "hala taşma riski var" denir

Problem nerede çıkıyor?

Linear queue'da front sürekli ilerler:

[10, 20, 30, 40, 50]

front $\rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Dequeue yaptıkça:

baş taraf boşalır ama front ileri gider

Sonuç: [, , 30, 40, 50]

rear en sonda

front ortada/ileride

Kritik sorun: “ekledikçe her yeri kullanamazsın”

[, , 30, 40, 50]

front = 2

rear = 4

Şimdi enqueue:

✗ 0 ve 1 boş ama kullanılamaz

Senin düşünce

Boş yer varsa kullanırım

Ekledikçe tüm alan dolar

Hafıza tam kullanılır

Gerçek

✗ her boş yer kullanılamaz

✗ sadece ileri doğru dolar

✗ fragmentation(parçalanma) olur

1) “Sınır kontrolü koyduk, neden hâlâ taşma riski var?”

Problem 1: Erken overflow + veri kaybı

kuyruk “boş” gibi yapılıyor

ama aslında içeride hâlâ kullanılabilecek yapı bilgisi var

reset “gerçek duruma göre değil”

sadece index'e bakıyor

Problem 2: “Gerçek boşluk” ile “mantıksal boşluk” karışır

reset yapmazsan:

baştaki boş alanlar **asla tekrar kullanılmaz**

reset yaparsan:

tüm yapı “sıfırlanmış gibi” olur

“Doğruluk neden zayıf?”

1. FIFO mantığı bozulabilir

indexler sıfırlanır

ama fiziksel veri dizide eski yerinde kalabilir

yeni eklemeler eski yapıyla karışabilir

2. “Gerçek durum ≠ temsil edilen durum”

queue’nun **gerçek durumu** ile senin tuttuğun front/rear/cnt uyumsuz hale gelebilir

Başlangıç

Dizi boyutu: `SIZE = 5`

Başlangıçta:

Index	0	1	2	3	4
Data					
Front	0				
Rear	-1				

- `front = 0` → kuyruk başı buradan çıkarılacak
- `rear = -1` → henüz eleman yok

Adım 1: `enqueue(&q, 10)`

`// C++`

```
q->rear++;           // rear = 0
q->data[rear] = 10
q->cnt++;            // cnt = 1
```

Index	0	1	2	3	4
Data	10				
Front	0				
Rear	0				

Adim 2: enqueue(&q, 20)

↗ C++

```
rear++;           // rear = 1  
data[1] = 20  
cnt++            // cnt = 2
```

Index	0	1	2	3	4
Data	10	20			
Front	0				
Rear	1				

Adim 3: enqueue(&q, 30)

↗ C++

```
rear++;           // rear = 2  
data[2] = 30  
cnt++            // cnt = 3
```

Index	0	1	2	3	4
Data	10	20	30		
Front	0				
Rear	2				

Adım 4: dequeue(&q)

⌞ C++

```
x = data[front] // x = 10
front++         // front = 1
cnt--           // cnt = 2
```

Index	0	1	2	3	4
Data	10	20	30		
Front	1				
Rear	2				

Adım 5: enqueue(&q, 40)

⌞ C++

```
rear++           // rear = 3
data[3] = 40
cnt++            // cnt = 3
```

Index	0	1	2	3	4
Data	10	20	30	40	
Front	1				
Rear	3				

Linear queue “temel olarak verimsiz”

Problem kodda değil, modelde:

dequeue → boşluk bırakır, enqueue → sadece sona gider

baştaki boşluklar ölür

Bu yüzden doğru çözüm-> Circular queue

Circular Queue (Dairesel Kuyruk)

```
#include <iostream>
using namespace std;
#define SIZE 5
```

```
struct CircularQueue {
    int data[SIZE];
    int front, rear;
    int cnt;
};
```

```
void initialize(CircularQueue *q) {
    q->front = 0;
    q->rear = -1;
    q->cnt = 0;
}
```

```
bool isFull(CircularQueue *q) {
    return q->cnt == SIZE;
}
```

```
bool isEmpty(CircularQueue *q) {
    return q->cnt == 0;
}
```

```
void enqueue(CircularQueue *q, int val) {
    if(isFull(q)) {
        cout << "Kuyruk dolu" << endl;
        return;
    }
    q->rear = (q->rear + 1) % SIZE;
    q->data[q->rear] = val;
    q->cnt++;
    cout << val << " kuyruğa eklendi" << endl;
}
```

```
void dequeue(CircularQueue *q) {
    if(isEmpty(q)) {
        cout << "Kuyruk boş" << endl;
        return;
    }
    int x = q->data[q->front];
    q->front = (q->front + 1) % SIZE;
    q->cnt--;
    cout << x << " kuyruktan silindi" << endl;
}
```

```
void print(CircularQueue *q) {
    if(isEmpty(q)) {
        cout << "Kuyruk boş" << endl;
        return;
    }
    int i = q->front;
    int c = q->cnt;
    while(c-- > 0) {
        cout << q->data[i] << " ";
        i = (i + 1) % SIZE;
    }
    cout << endl;
}
```

```
int main() {
    CircularQueue q;
    initialize(&q);
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    print(&q);
    dequeue(&q);
    enqueue(&q, 40);
    enqueue(&q, 50);
    enqueue(&q, 60); // Kuyruk doldu
    print(&q);
    dequeue(&q);
    enqueue(&q, 70);
    print(&q);
}
```

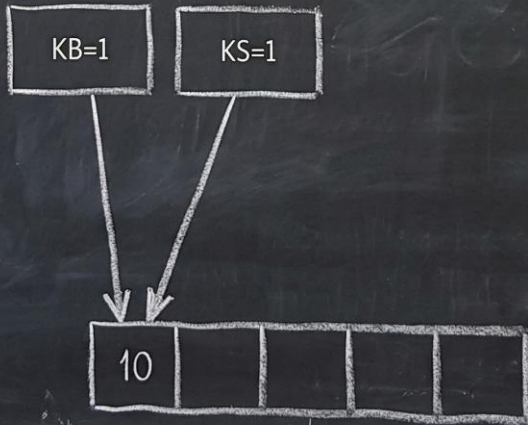
DİZİLERDE Doğrusal kuyruk yapısında bellek verimli kullanılmazken bu sorun bir ölçüde dairesel kuyrukla çözülmüştür. Bu kısım derste detaylandırılacaktır!!

Örnek 1: $A[i]=\{10, 20, 30, 40, 50, 60, 70\}$ dizisinin elemanlarını dairesel kuyruk veri yapısı ile belleğe yerleştirelim.

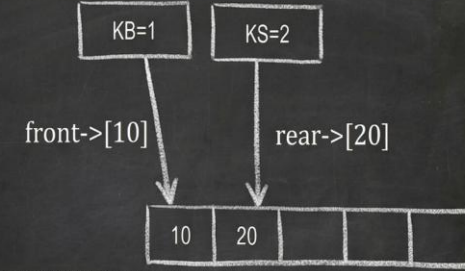
Başlangıç durumunda KB, KS ve dizinin durumu.



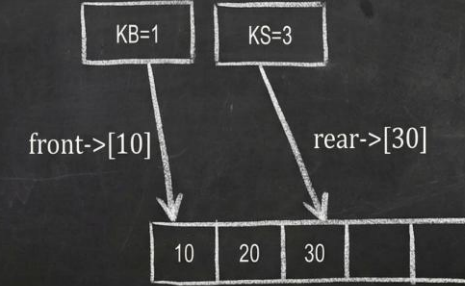
10 elemanını ekleyelim.



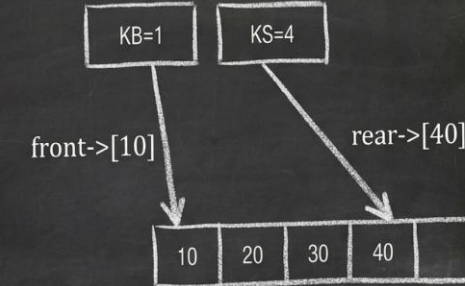
20 elemanını ekleyelim.



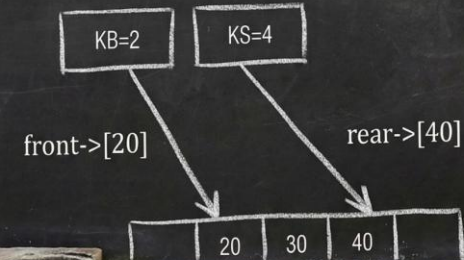
30 elemanını ekleyelim.



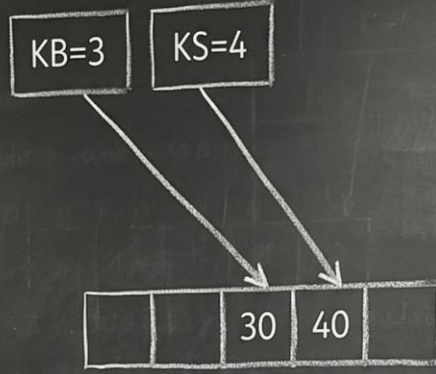
40 elemanını ekleyelim



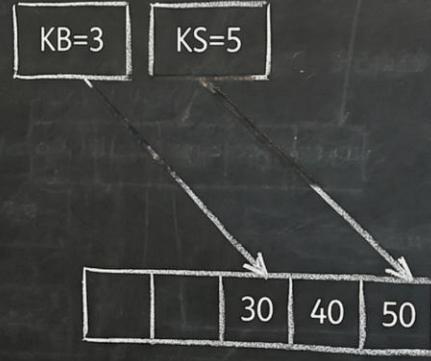
Bir eleman çıkaralım.



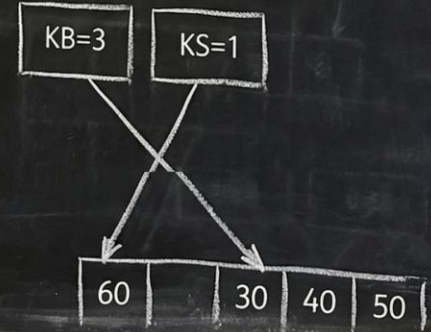
Bir eleman çıkaralım



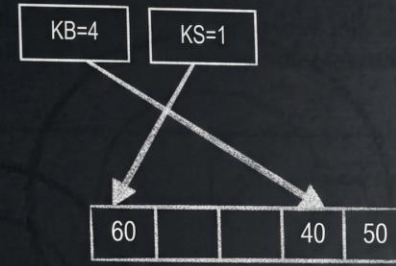
50 elemanını ekleyelim



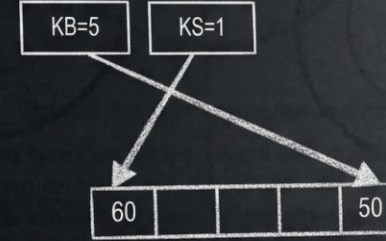
60 elemanını ekleyelim. (KS=5) = es Bu durumda KS=1 yapılarak baş taraftaki boş bellek gözlerinin kullanılması sağlanır.



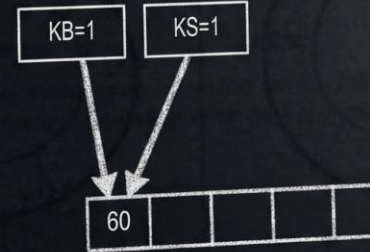
Bir eleman çıkaralım



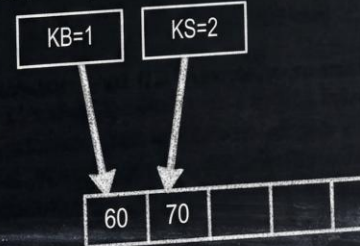
Bir eleman çıkaralım



Bir eleman çıkaralım. KB=KS 'dur. Adım 4 gereği (if KB=es KB=1 else KB=KB+1 // Daireselliğin oluşturulduğu kısım) KB=1 yapacağız ve KB değışkeni için de bir daire oluşturduk.



Şimdi 70 elemanını ekleyelim.



Dizi boyutunu aşacak surette veri ekleme işlemi yapılıyorsa dizinin dinamik formatta yeniden oluşturulması gerekir. Ya da diziye eklenen elemanların ardı sıra silinmesi sonucunda da dizi boyutunun dinamik formatta yeniden oluşturulması gerekir.

Tercihe bağlı olarak bu hedefin kodlamasını yapabilirsiniz..

C++ dilinde liste yapısı ile oluşturulan LİNEAR kuyruk kodlaması...

```
#include <iostream>
using namespace std;

struct node {
    int veri;
    node* next;
};

struct kuyruk {
    node* front;
    node* rear;
    int toplam;
};

// Kuyruğu başlangıç değerlerine getiren fonksiyon
void ata(kuyruk* q) {
    q->front = NULL;
    q->rear = NULL;
    q->toplam = 0;
}

void ekle(kuyruk* q, int veri) {
    node* yeni = new node();
    yeni->veri = veri;
    yeni->next = NULL;

    if (q->toplam == 0) {
        // Kuyruk boşsa, yeni eleman hem baş hem sondur
        q->front = q->rear = yeni;
    } else {
        // Kuyrukta eleman varsa, mevcut sonuncunun sonuna bağla
        q->rear->next = yeni;
        q->rear = yeni; // Arka işaretçisini güncelle
    }
    q->toplam++;
    cout << veri << " kuyruğa eklendi." << endl;
}
```

```
void sil(kuyruk* q) {
    if (q->toplam == 0) {
        cout << "Kuyruk bos, silinecek eleman yok!" << endl;
        return;
    }

    node* temp = q->front; // Silinecek olanı yedekle
    q->front = q->front->next; // Başı bir sonrakine kaydır

    cout << temp->veri << " kuyruktan silindi." << endl;
    delete temp; // Belleği temizle
    q->toplam--;

    // Eğer son elemanı sildiysek, rear'ı da NULL yapmalıyız
    if (q->toplam == 0) {
        q->rear = NULL;
    }
}

void yazdir(kuyruk* q) {
    if (q->toplam == 0) {
        cout << "Kuyruk bos." << endl;
        return;
    }

    node* temp = q->front;
    cout << "Kuyruk durumu: ";
    while (temp != NULL) {
        cout << temp->veri << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

struct kuyruk → veri tipi oluşturur.
kuyruk rootk; → bu tipten bir değişken oluşturur.
rootk.front, rootk.rear, rootk.data[i] →
değişkenin üyelerine erişir.
Fonksiyonlar pointer ile alırsa &rootk gönderilir.
```

```
int main() {
    kuyruk rootk;
    ata(&rootk);

    ekle(&rootk, 10);
    ekle(&rootk, 20);
    ekle(&rootk, 30);
    yazdir(&rootk); // 10 -> 20 -> 30 -> NULL

    sil(&rootk); // 10 silinir
    yazdir(&rootk); // 20 -> 30 -> NULL

    ekle(&rootk, 40);
    yazdir(&rootk); // 20 -> 30 -> 40 -> NULL

    return 0;
}
```

ata(&rootk) ifadesinde & (adres operatörü) kullanılmasının sebebi, rootk değişkeninin adresini fonksiyona geçirmek ve böylece fonksiyon içinde değiştirilen değer, çağrıldığı yerde de güncellenmesini sağlamaktır.

ÖZETLEYELİM....

Sabit Dizi (Array) ile Kuyruk

Eğer kuyruk **sabit boyutlu bir diziyle** uygulanmışsa:

Kuyruktan eleman çıkardıkça bas indeksi artar.

Ama dizinin başındaki alanlar boş kalsa bile kullanılamaz hâle gelir.

Bellek verimsiz kullanılır çünkü kuyruğun başı boşalsa da tekrar kullanılmaz

Çözüm: Dairesel kuyruk (circular queue) mantığıyla bas ve son indeksleri modüler olarak döndürülür

Böylece eski boş alanlar tekrar kullanılır

Liste ile Kuyruk (Linked List)

Listeyle yapılan kuyrukta:

Eleman silindiğinde (örneğin baştan dequeue yapıldığında), o düğümün pointer'ı free() ile silinirse bellek geri kazanılır.

Yani bu yapıda bellek geri kazanımı mümkündür, ama silinen düğüm free() ile silinmezse bellek sızıntısı (memory leak) oluşur.

Yani:→ free(çıkarılan_düğüm) yaptıysan: bellek geri kazanılır→ Yapmadıysan: kullanılmayan ama dolu olan bellek oluşur

Öncelikli Kuyruk C++ ile İşletimi

1. Öncelikli Kuyruk Nedir?

Öncelikli kuyruk (priority queue), klasik bir kuyruğa benzeyen ama **her elemanın bir öncelik değeri olduğu** bir veri yapısıdır. Temel mantığı:

Her elemanın bir **öncelik değeri** vardır.

Elemanlar, **öncelik sırasına göre** kuyrukta tutulur.

Ekleme sırasında, eleman **doğru pozisyona** yerleştirilir.

Çıkarma (dequeue) genellikle **en yüksek öncelikli elemandan** yapılır.

Örnek: Hastane acil servisi

Öncelikli kuyruk: hastalar önceliklerine göre sıraya konur.

Öncelik: kritik durum → yüksek değer, normal → düşük değer.

2. Dairesellik Mantığı

Dairesel bir liste:

Listenin son elemanı, NULL yerine **baş a işaret eder**, yani liste bir halka oluşturur.

Böylece listenin başından sonuna ya da sonundan başına kolayca geçilebilir.

Liste boşsa: ilk eleman **kendisine işaret eder**.

Liste doluysa: yeni ekleme uygun yere yapılır, dairesellik korunur.

Dairesel yapı sayesinde:

Listenin sonunda dolaşırken NULL kontrolü yerine ilk kontrolü yapılır.

Kuyruk başı veya sonu değiştirilse bile, halka bozulmaz.

```

#include <iostream>
using namespace std;

// Hasta sınıfı: Her hastanın adı ve önceliği var
class Hasta {
public:
    string isim;
    int oncelik;

    Hasta(string i, int o) {
        isim = i;
        oncelik = o;
    }
};

// Liste elemanı (node) sınıfı: Hasta verisi ve sonraki pointer
class SiraEleman {
public:
    Hasta* veri;
    SiraEleman* sonraki;

    SiraEleman(Hasta* h) {
        veri = h;
        sonraki = nullptr;
    }
};

```

```

// Öncülük kuyruğu sınıfı: Dairesel bağlı liste mantığı
class Sira {
private:
    SiraEleman* ilk; // Listenin başı

public:
    Sira() {
        ilk = nullptr;
    }

    // Hasta ekleme fonksiyonu
    void ekle(Hasta* yeniHasta) {
        SiraEleman* yeni = new SiraEleman(yeniHasta);

        // 1. Liste boşsa
        if (ilk == nullptr) {
            ilk = yeni;
            ilk->sonraki = ilk; // Dairesel liste
            return;
        }

        // 2. Yeni hasta en yüksek öncelikli ise (başına ekle)
        if (ilk->veri->oncelik < yeniHasta->oncelik) {
            SiraEleman* iter = ilk;
            // Listenin sonuna git (son elemanın 'sonraki' ilk'i işaret eder)
            while (iter->sonraki != ilk) {
                iter = iter->sonraki;
            }
            iter->sonraki = yeni;
            yeni->sonraki = ilk;
            ilk = yeni; // Baş artık yeni hasta
            return;
        }

        // 3. Araya veya sona ekleme
        SiraEleman* iter = ilk;
        while (iter->sonraki != ilk && iter->sonraki->veri->oncelik > yeniHasta->oncelik) {
            iter = iter->sonraki;
        }
        // Yeni elemanı bulunduğu pozisyona ekle
        yeni->sonraki = iter->sonraki;
        iter->sonraki = yeni;
    }
}

```

```
// Listeyi ekrana yazdır (daireysel olduğu için ilk'e kadar dön)
void yazdir() {
    if (ilk == nullptr) {
        cout << "Liste bos." << endl;
        return;
    }

    SiraEleman* iter = ilk;
    do {
        cout << "[" << iter->veri->oncelik << ":" << iter->veri->isim << "]" -> ";
        iter = iter->sonraki;
    } while (iter != ilk);
    cout << "(geri donus: " << ilk->veri->isim << ")" << endl;
}

};

int main() {
    Sira kuyruk;

    // Hastaları ekleyelim
    kuyruk.ekle(new Hasta("Mehmet", 10));
    kuyruk.ekle(new Hasta("Ayse", 5));
    kuyruk.ekle(new Hasta("Ali", 8));
    kuyruk.ekle(new Hasta("Zeynep", 12)); // En yüksek öncelik

    kuyruk.yazdir();

    return 0;
}
```

Kodun Çalışma Mantığı

- 1.Liste Boşsa** → ilk hasta kendisine işaret eder.
- 2.Yeni hasta başa mı eklenmeli?** → en yüksek öncelik varsa başa ekle, son elemanın işaretini yeni başa bağla.
- 3.OrTaya veya sona ekle** → sırayla ilerle, doğru pozisyonu bul, yeni elemanı ekle.
- 4.Yazdırma** → dairesel listeyi baştan dönerek gösterir.

Hastaların eklenme sırası ve öncelikleri

- 1."Mehmet", 10 → Liste boş, başa eklenir → [10:Mehmet]
- 2."Ayse", 5 → 10 > 5 → araya/sona ekleme → [10:Mehmet] -> [5:Ayse]
- 3."Ali", 8 → 10 > 8, 5 < 8 → Ali 10 ile 5 arasına eklenir → [10:Mehmet] -> [8:Ali] -> [5:Ayse]
- 4."Zeynep", 12 → 12 > 10 → başa eklenir → [12:Zeynep] -> [10:Mehmet] -> [8:Ali] -> [5:Ayse]
Liste dairesel olduğu için son eleman (Ayse) baş (Zeynep)'e geri döner.

[12:Zeynep] -> [10:Mehmet] -> [8:Ali] -> [5:Ayse] -> (geri donus: Zeynep)

Kaynakca

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel,Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++Cve Java ile, Sadi Evren Şeker

<https://chatgpt.com/>

<https://gemini.google.com/>

<https://bilgisayarkavramlari.com/2010/12/30/dairesel-bagli-liste-ile-oncelige-sahip-hasta-sirasi-takibi/>

<https://www.bilgisayarkavramlari.com/wp-content/uploads/circhasta.cpp>